

SOLID et architecture hexagonale

Concevoir une application qui évolue
Anaël Chardan · IMT Atlantique · 2026

Le service : enregistrer une partie

DEMANDE

Brass, 2 à 4 participants

Brass, 1 ou 5 participants

Jeu absent du catalogue

RÉSULTAT ATTENDU

Partie valide

Refus métier

Jeu introuvable

La même politique doit s'appliquer à chaque entrée.

Trois évolutions du même service

ÉVOLUTION

Catalogue local remplacé par
BGG

Stockage persistant

Traitement depuis une CLI

CONTRAINTE À CONSERVER

Règles métier identiques

Aucun enregistrement invalide

Aucune duplication des règles

Les critères de conception

CRITÈRE

Impact d'un changement

Testabilité

Complexité ajoutée

QUESTION D'INGÉNIERIE

Quels modules faut-il modifier ?

Quelles dépendances faut-il
démarrer ?

Quel coût ajoute la séparation ?

Une abstraction se justifie par le besoin qu'elle protège.

Le point de départ couplé

```
const { boardgameName, players } = JSON.parse(json);
const game = { name: "Brass", min: 2, max: 4 };
if (players.length < game.min ||
    players.length > game.max) throw new Error();
const row = {
  game: game.name,
  players_json: JSON.stringify(players)
};
```

Transport, politique métier et représentation du stockage.

SOLID : cinq critères de conception

PRINCIPE

Single Responsibility

Open / Closed

Liskov Substitution

Interface Segregation

Dependency Inversion

PROPRIÉTÉ RECHERCHÉE

Cohésion des raisons de changer

Extension sur un axe identifié

Compatibilité des
comportements

Contrats adaptés aux
consommateurs

Dépendances vers les besoins
métier

SRP : les axes de changement

Origine du changement

Client HTTP



Décodage
de la requête

Règles du jeu



Validation
de la partie

Schéma SQL



Conversion
pour le stockage

Un module peut coordonner plusieurs opérations cohérentes.

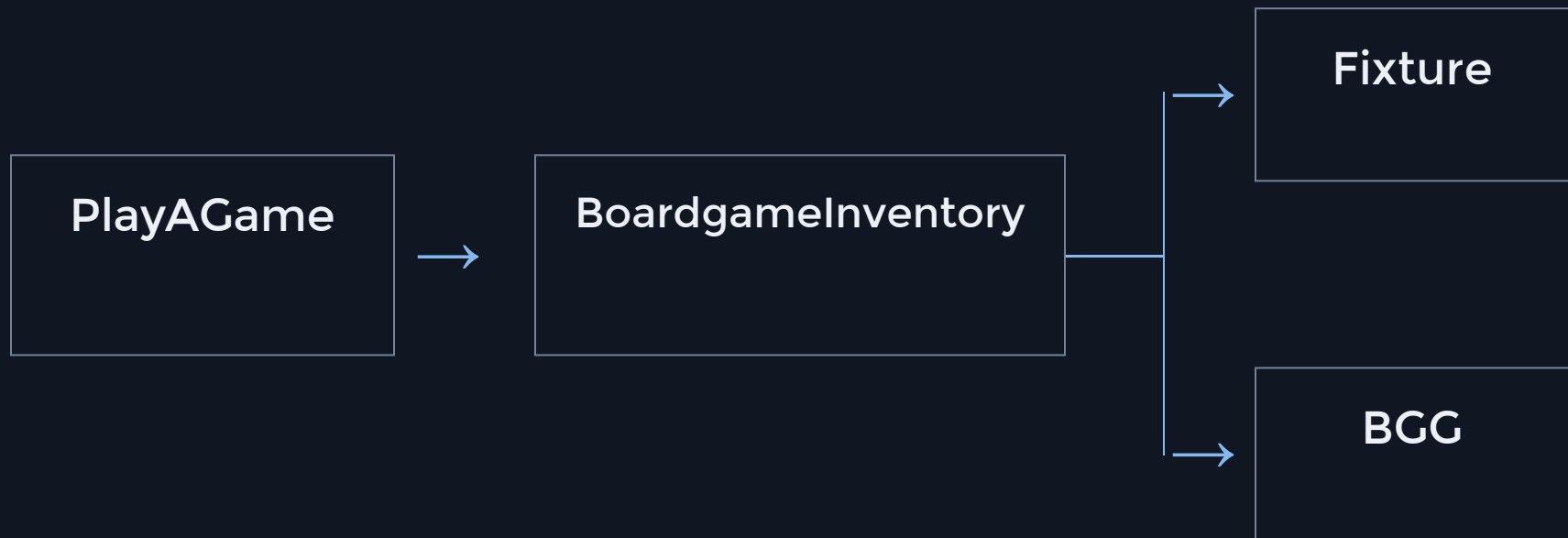
Validation indépendante du transport

```
function validateCount(  
  game: Game, players: string[]  
) {  
  if (players.length < game.min ||  
      players.length > game.max) {  
    throw new Error("Nombre invalide");  
  }  
}
```

Le test porte sur la règle et ses données d'entrée.

OCP : remplacer le catalogue

Deux adaptateurs possibles pour un même port



Le câblage choisit un adaptateur. Le cas d'usage reste identique.

Le câblage concentre le changement

```
// Configuration locale
const play = buildPlayAGame(fixtureCatalogue, store);

// Configuration avec le fournisseur externe
const play = buildPlayAGame(bggCatalogue, store);
```

Une nouvelle politique métier peut encore modifier le cas d'usage.

LSP : le contrat observable du catalogue

SITUATION

Jeu connu

Jeu inconnu

Fournisseur indisponible

COMPORTEMENT DU PORT

Boardgame avec des limites
valides

null

CatalogUnavailable

Résultat, absence et erreurs font partie du contrat.

Une signature compatible ne suffit pas

```
const brokenCatalogue: Catalogue = {  
  async find(name) {  
    throw new Error("Jeu inconnu");  
  }  
};
```

Le client attend null pour une absence, pas une exception.

Le contrat commun des catalogues

```
expect(await catalogue.getBoardgameByName("Inconnu"))  
  .toBeNull();  
  
expect(await catalogue.getBoardgameByName("Azul"))  
  .toMatchObject({ minNumberOfPlayers: 2 });
```

La même suite de contrat s'applique aux deux catalogues.

ISP : le contrat vu par le consommateur

```
interface PlayRepository {  
    save(play: Play): Promise<void>;  
    all(): Promise<Play[]>;  
    deleteAll(): Promise<void>;  
    rebuildStatistics(): Promise<void>;  
}
```

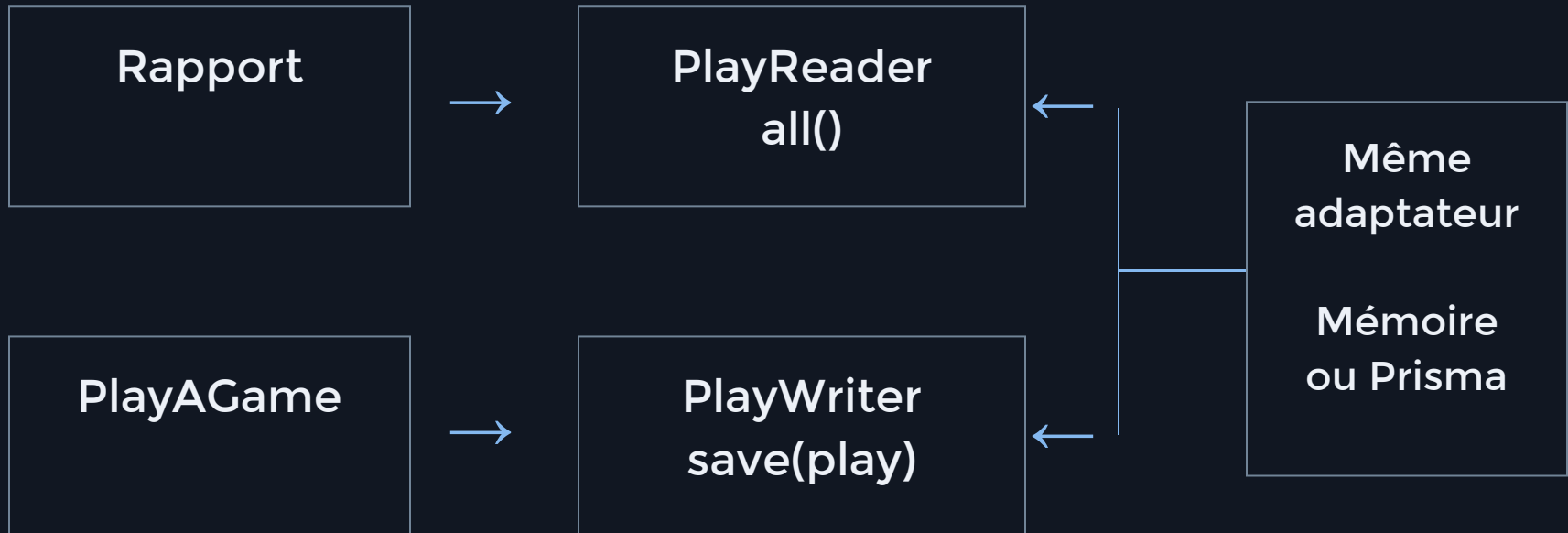
Un rapport de lecture n'utilise qu'une partie de ce contrat.

Des ports distincts pour lire et écrire

UTILISE

CONTRATS

IMPLÉMENTE



Chaque consommateur utilise uniquement le contrat nécessaire.

DIP : le métier dépend du stockage

```
// Module métier couplé à une bibliothèque
import { PrismaClient } from "@prisma/client";

const db = new PrismaClient();
await db.play.create({ data: sqlRow });
```

La politique métier connaît ici Prisma et la forme SQL.

Le domaine définit le port

```
// Contrat défini dans domain/ports.ts
interface PlayWriter {
  save(play: Play): Promise<void>;
}

// Instance choisie dans composition.ts
const play = buildPlayAGame(catalogue, store);
```

DIP : direction des dépendances. Injection : assemblage des objets.

Les décisions obtenues avec SOLID

DÉCISION

Isoler la validation

Définir le contrat du catalogue

Distinguer lecture et écriture

Placer les ports dans le domaine

EFFET ATTENDU

Tester la politique sans transport

Remplacer sa provenance

Réduire les exigences des clients

Éviter les imports techniques

Contrepartie : des interfaces et des conversions à maintenir.

Couches et frontière applicative

Organisation en couches

Ports et adaptateurs

Répartir les responsabilités

Définir les échanges avec l'extérieur

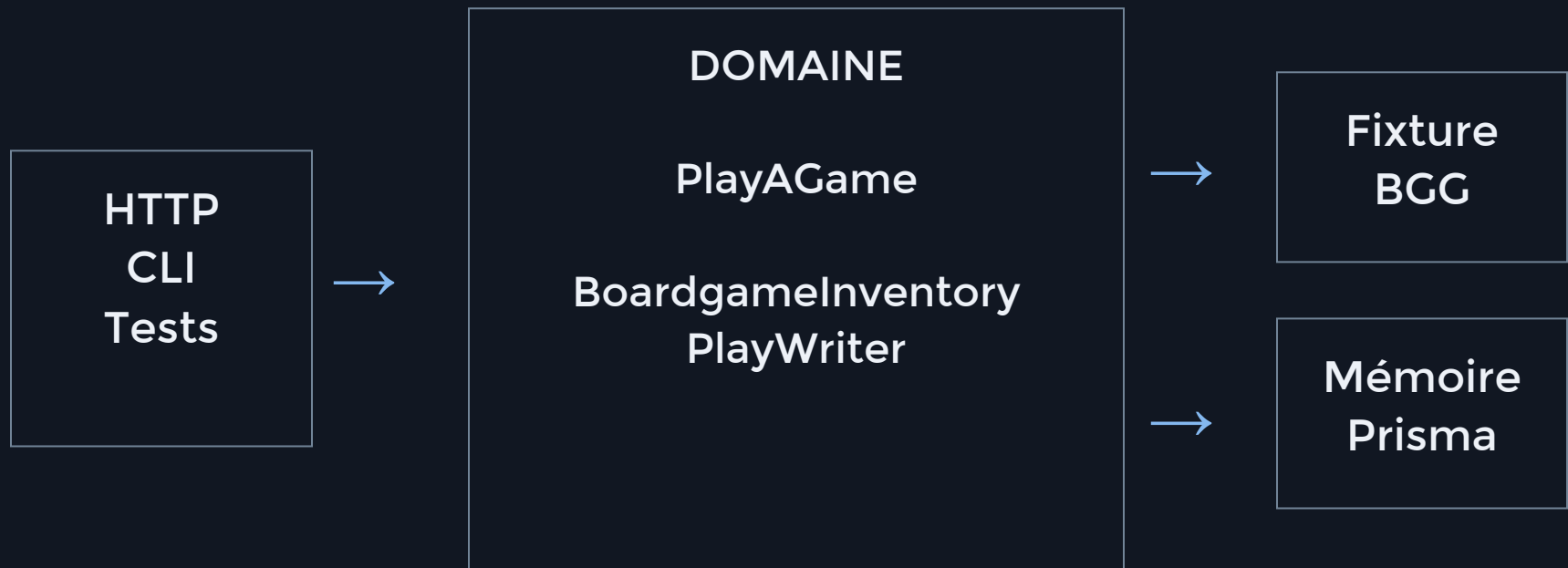
Rendre la structure lisible

Rendre les technologies substituables

Les deux approches peuvent coexister.

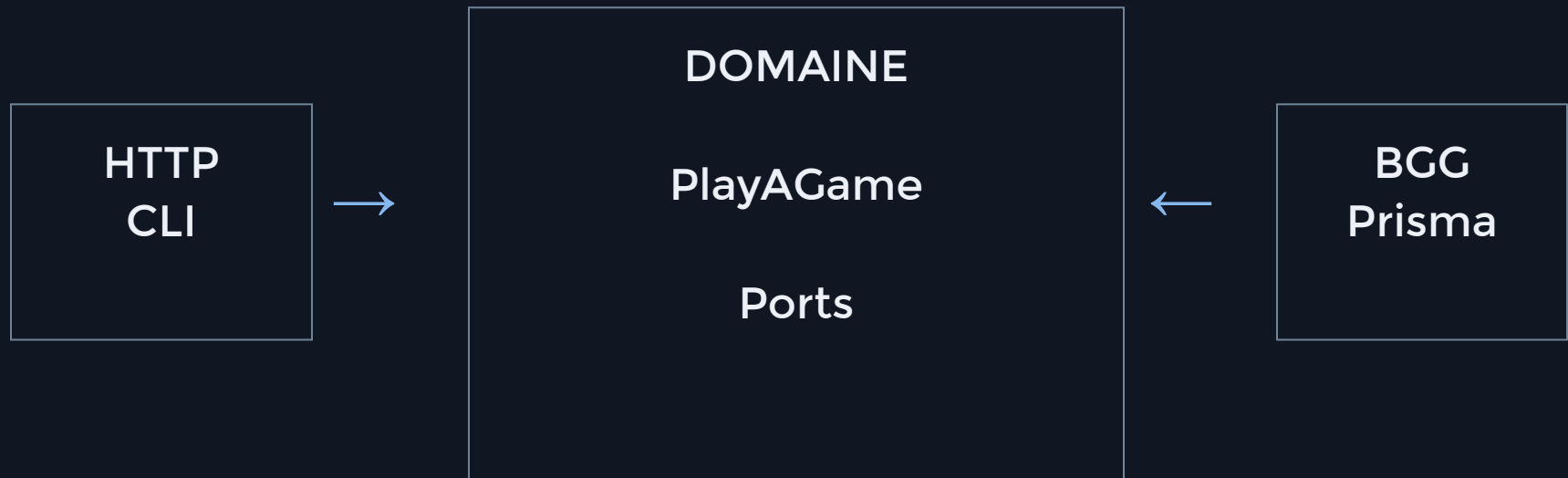
La frontière du domaine

Conversations à l'exécution



Dépendances à la compilation

Chaque flèche représente un import de type



Les adaptateurs importent les contrats définis dans le domaine.

Séquence d'un enregistrement

Ordre des appels dans la version avec sauvegarde

HTTP

PlayAGame

Catalogue

Writer

1 Demande →

2 Recherche →

3 Validation

4 Enregistrement →

Le sens des appels diffère du sens des dépendances du code.

Le modèle manipulé par le cas d'usage

```
type Boardgame = {  
    name: string; bggId: string;  
    minNumberOfPlayers: number;  
    maxNumberOfPlayers: number;  
};  
type Play = {  
    boardgameName: string; bggId: string;  
    players: string[];  
};
```

La validation dans le cas d'usage

```
const game = await inventory.getBoardgameByName(name);  
if (!game) throw new BoardgameNotFound();  
  
if (count < game.minNumberOfPlayers ||  
    count > game.maxNumberOfPlayers) {  
    throw new InvalidPlayerCount(min, max);  
}
```

À cette étape, une partie valide est retournée sans écriture.

Les tests aux frontières de l'intervalle

PARTICIPANTS

1

2 et 4

5

Jeu inconnu

RÉSULTAT

Refus

Acceptation des deux bornes

Refus

Erreur distincte du nombre
invalide

Mutation de la condition : $>$ devient $\geq$.

Pause

Reprise dans 10 minutes

Intégration des adaptateurs HTTP, catalogue et stockage.

HTTP : validation et traduction

```
const parsed = requestSchema.safeParse(request.body);  
if (!parsed.success) {  
  return reply.code(400).send({  
    error: "Requête invalide"  
  });  
}  
const result = await play(parsed.data);  
return reply.code(200).send(result);
```

Étape 04 : validation seule. Étape 06 : création après écriture.

La politique d'erreur de l'API

SITUATION	RÉPONSE CHOISIE
Demande mal formée	400
Jeu introuvable	404
Règle métier non satisfaite	422
Catalogue indisponible	503
Erreur interne inattendue	500

Les codes HTTP appartiennent à l'adaptateur.

Absence métier ou panne technique

Absence métier

Recherche valide, aucun jeu

Le port retourne null

Notre API répond 404

Défaillance technique

Authentification ou réseau en échec

Le port lève
CatalogUnavailable

Notre API répond 503

Intégration du catalogue BGG

CONTRAT STABLE

Obtenir un Boardgame

Signaler une absence

Signaler une défaillance

CHOIX DE L'ADAPTATEUR

Données locales ou service BGG

null

Erreur technique distincte

Le mode BGG exige un jeton d'application valide.

La traduction au niveau de l'adaptateur

DOCUMENT EXTERNE

```
<minplayers value="2" />  
<maxplayers value="4" />
```



OBJET MÉTIER

```
minNumberOfPlayers: 2  
maxNumberOfPlayers: 4
```

L'adaptateur valide le document et produit le type métier.

Les niveaux de vérification

TEST

Métier

Contrat

Adaptateur BGG

Intégration SQL

CE QU'IL ÉTABLIT

Validité des règles et de leurs bornes

Compatibilité des comportements

Conversion de réponses contrôlées

Écriture et relecture dans une base

Un faux transport ne vérifie ni BGG en direct ni ses identifiants.

La sauvegarde fait partie du cas d'usage

```
// Les validations ont réussi
const play = { boardgameName, bggId, players };

await writer.save(play);
return play;
```

La réponse de création attend le succès de l'écriture.

Deux propriétés à vérifier

Partie refusée

Écriture en échec

Aucun enregistrement

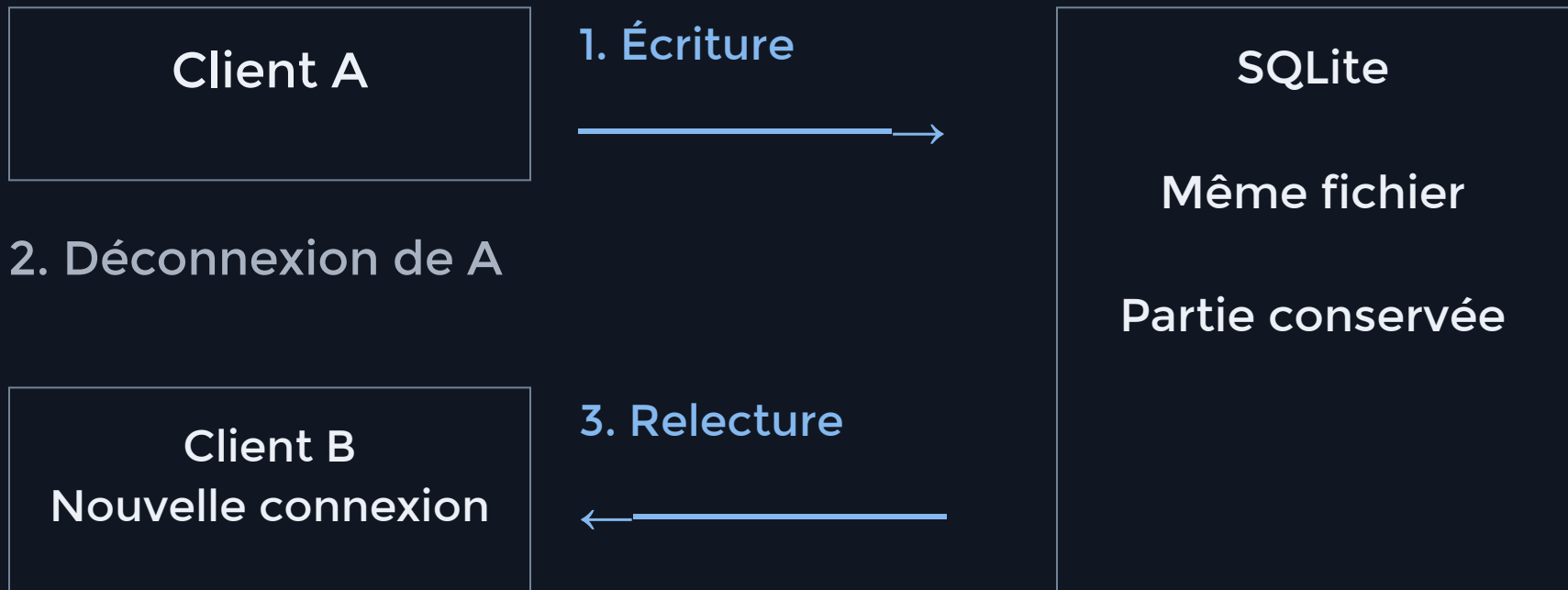
Aucune réponse de création

Le stockage reste inchangé

L'erreur remonte à
l'adaptateur

Vérifier les effets observables, pas seulement le retour du cas d'usage.

Persistance après reconnexion



La partie reste disponible après la fermeture du premier client.

Une entrée CLI sur le même cas d'usage

```
npm run cli -- "Azul" Alice Bob

// Adaptateur CLI
await play({ boardgameName, players });
```

HTTP et CLI partagent le stockage lorsqu'ils ciblent la même base.

Unicité des participants

DEMANDE

Alice, Bob

Alice, Alice

Alice, « alice » avec espaces

RÉSULTAT ATTENDU

Acceptation

Refus

Refus après normalisation

Convention du modèle : le nom normalisé représente l'identité.

Une règle commune aux deux entrées

```
const names = players.map(name => name.trim());  
const unique = new Set(  
    names.map(name => name.toLowerCase())  
);  
if (unique.size !== names.length) {  
    throw new InvalidParticipants();  
}
```

Le diff 08 / 09 ajoute la règle et les vérifications HTTP et CLI.

Le coût des frontières

BÉNÉFICE RECHERCHÉ

Tests métier isolés

Adaptateur remplaçable

Réutilisation du cas d'usage

COÛT À ACCEPTER

Contrats à concevoir

Conversions à vérifier

Parcours du code moins direct

Le contexte détermine si cette séparation est rentable.

Les limites du modèle et la suite

Identité : gérer les homonymes avec des identifiants.

Fiabilité : éviter un double enregistrement lors d'un retry.

Cohérence : définir la transaction si plusieurs écritures doivent réussir ensemble.

Ces besoins doivent précéder les nouvelles abstractions.